is not foolproof. As we saw earlier, tools like curl could be used to spoof the origin of a request. If you want stronger protection, you will need a *CSRF token*.

A simple definition is that a CSRF token is an unguessable secret shared between the client and the server. If a CSRF token is invalid, the request fails. Appendix C goes into more details about CSRF tokens and how they are used.

There are many packages available for adding CSRF token support to same-origin requests. Appendix C also includes an example that uses the CSURF middleware package to add CSRF token support to a sample server. This is easier to do on same-origin requests, where the client and server are the same machine, or share the same code.

Implementing CSRF token support is a bit trickier for cross-origin requests because in order for CSRF tokens to work, the server and client must agree upon a CSRF token format. A cross-origin request's client and server may live on separate servers and have separate codebases, which makes it hard to sync secrets. Another issue is that the CSRF token usually has embedded user information, such as a user ID or session ID. On same-origin requests, the user info can be derived from the cookie. On cross-origin requests, the client doesn't have access to the server's cookies.

These issues make it difficult to add CSRF protection to CORS requests. One possible way to do this is to have the client embed an iframe from the server. The server can place the CSRF token in this iframe, and then give it to the client via `postMessage`.

Security is very difficult to get right. If you need CSRF protection on CORS requests, here are a few things to keep in mind:

- *Consider whether you need CSRF protection.* CSRF protection is only needed if you are requesting protected data that includes the cookie.
- *Validate the Origin header.* This is a good form of CSRF protection, and it may be sufficient for your needs. While tools such as curl can spoof the Origin header, spoofing along with the cookie is harder (curl wouldn't have access to the cookie). Older browsers that allow the Origin header to be set don't support CORS.
- *Consider same-origin requests instead.* If you have a particular feature that requires CSRF protection, such as posting a new weblog, consider making it a same-origin feature instead. Same origin requests have proven mechanisms for protecting against CSRF. Part of using CORS successfully is understanding its limits, and you may save yourself a headache or two by using same-origin requests in this particular case.
- *Use something other than a cookie to validate the user.* If you're building a public API, or need to provide authorized access to all origins, an authorization mechanism like OAuth2 might be a better fit for your needs.

The next section explores the last bullet point in-depth by looking at what OAuth2 is and how it works.